



Università degli Studi di Bergamo

Dipartimento di Ingegneria

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025/2026

REQUISITI

Ingegneria del Software

The Scouting Suite

Brivio Andrea — Matricola: 1089359

Fischetti Alessandro — Matricola: 1080491

Pesenti Lorenzo — Matricola: 1072635

Indice

1	Introduzione	2
1.1	Scopo del Documento	2
1.2	Contesto del Progetto	2
1.3	Glossario e Terminologia	2
2	Requirements Engineering	2
2.1	Elicitazione dei Requisiti	2
2.2	Classificazione dei Requisiti	3
3	Requisiti Funzionali	3
3.1	RF-01: Gestione Dataset (ETL Pipeline)	3
3.2	RF-02: Visualizzazione Interfaccia Utente	4
3.3	RF-03: Filtraggio Base	4
3.4	RF-04: Filtraggio Statistico Avanzato	5
3.5	RF-05: Sistema di Colonne Dinamiche	5
3.6	RF-06: Apertura Automatica Browser	6
4	Requisiti Non Funzionali	6
4.1	RNF-01: Usabilità	6
4.2	RNF-02: Prestazioni	6
4.3	RNF-03: Affidabilità	6
4.4	RNF-04: Manutenibilità	7
4.5	RNF-05: Portabilità	7
4.6	RNF-06: Sicurezza (Future Enhancement)	7
5	Requisiti di Dati	7
5.1	RD-01: Struttura Dataset	7
5.2	RD-02: Modello Entity	8
5.3	RD-03: Persistenza	8
6	Vincoli di Sistema	8
6.1	VS-01: Tecnologici	8
6.2	VS-02: Sviluppo	8
6.3	VS-03: Architetture	9
7	Software Quality	9
7.1	Obiettivi di Qualità	9
7.2	Relazione tra Qualità e Progettazione	9
8	Modellazione dei Requisiti	9
8.1	Diagrammi UML Prodotti	9
8.2	Tracciamento Requisiti	10
9	Conclusioni	10

1 Introduzione

1.1 Scopo del Documento

Questo documento definisce i requisiti funzionali e non funzionali del progetto **Scouting Suite**, un sistema completo per l'analisi statistica avanzata di calciatori professionisti. L'obiettivo è fornire una specifica verificabile che guidi lo sviluppo, il testing e la validazione del sistema secondo gli standard dell'Ingegneria del Software.

1.2 Contesto del Progetto

Scouting Suite è un'applicazione web enterprise sviluppata con Spring Boot e Vaadin che permette:

- Importazione automatica di dataset calcistici (100+ metriche per giocatore)
- Filtraggio dinamico e combinabile su attributi statistici complessi
- Visualizzazione configurabile tramite interfaccia utente interattiva
- Analisi comparativa per lo scouting professionale

1.3 Glossario e Terminologia

Termine	Definizione
Dataset	File CSV strutturato contenente statistiche di giocatori (es. Season 2025-26/Player_Final.csv)
Entity Player	Oggetto JPA che rappresenta un calciatore con 100+ attributi statistici
PlayerRepository	Componente Spring Data JPA per l'accesso ai dati
ScoutingService	Servizio che implementa la logica di business principale
Specification	Pattern JPA per la costruzione dinamica di query
StatFilterCriteria	Oggetto DTO che rappresenta un filtro statistico (campo, min, max)
BrowserLauncher	Componente che automatizza l'apertura del browser all'avvio
MainView	Interfaccia utente principale sviluppata con Vaadin
ETL Pipeline	Processo Extract-Transform-Load per il caricamento dati

Tabella 1: Glossario dei termini tecnici

2 Requirements Engineering

2.1 Elicitazione dei Requisiti

I requisiti sono stati eliciti attraverso:

- **Analisi del dominio:** Studio delle metriche calcistiche moderne (xG, xAG, SCA, GCA, etc.)
- **Benchmarking:** Analisi di sistemi esistenti (Wyscout, Opta, FBref)

- **Prototipazione iterativa:** Feedback su interfaccia utente e usabilità
- **Vincoli accademici:** Conformità con il corso di Ingegneria del Software
- **Pattern recognition:** Identificazione di pattern architetturali appropriati

2.2 Classificazione dei Requisiti

Categoria	Descrizione
Requisiti Funzionali	Cosa il sistema deve fare (azioni e comportamenti)
Requisiti Non Funzionali	Come il sistema deve farlo (qualità, vincoli)
Requisiti di Dati	Struttura, formato e persistenza dei dati
Vincoli di Sistema	Limitazioni tecnologiche e architetturali

Tabella 2: Classificazione dei requisiti

3 Requisiti Funzionali

3.1 RF-01: Gestione Dataset (ETL Pipeline)

Descrizione: Il sistema deve caricare automaticamente un dataset CSV all'avvio dell'applicazione, convertendo i dati in oggetti Player e persistendoli in database H2.

Attori coinvolti: Sistema (automatico)

Pre-condizioni:

- File CSV presente in `src/main/resources/Season 2025-26/Player_Final.csv`
- Database H2 disponibile

Post-condizioni:

- Dati convertiti e persistiti correttamente
- Log del processo disponibile

Criteri di accettazione:

- Gestione valori nulli/"NaN" nei dati sorgente
- Conversione corretta dei tipi di dato (String, Integer, Double)
- Performance: caricamento di 1000+ record in <10 secondi
- Rollback in caso di errori critici

Componenti: `DataProcessingService`, `PlayerRepository`

3.2 RF-02: Visualizzazione Interfaccia Utente

Descrizione: Il sistema deve fornire un'interfaccia web responsive con grid dati configurabile, pannello filtri e sistema di colonne dinamiche.

Attori coinvolti: Scout, Analista

Criteri di accettazione:

- Grid con 100+ colonne organizzate per categorie
- Sistema di colonne selezionabili (MultiSelectComboBox)
- Record count dinamico
- Supporto responsive (adattamento a diverse risoluzioni)
- Tempo di rendering iniziale <3 secondi

Componenti: MainView, Grid<Player>, Column configurations

3.3 RF-03: Filtraggio Base

Descrizione: Il sistema deve permettere il filtraggio su attributi anagrafici e categoriali dei giocatori.

Campi supportati:

- Età (range: min-max)
- Nome (ricerca parziale case-insensitive)
- Squadra (ricerca parziale)
- Nazione (selezione da lista)
- Posizione (selezione da lista)
- Competizione (selezione da lista)

Criteri di accettazione:

- Aggiornamento in tempo reale (max 500ms)
- Combinazione di filtri multipli (AND logic)
- Reset completo dei filtri
- UI/UX intuitiva con validazione input

Componenti: ScoutingController, ScoutingService, PlayerRepository

3.4 RF-04: Filtraggio Statistico Avanzato

Descrizione: Il sistema deve supportare filtri dinamici su attributi statistici numerici con operatori di range.

Campi supportati:

- Goals, Assists, xG, xAG, npG
- Shots, Shots on Target, Key Passes
- Tackles, Interceptions, Clearances
- E 80+ altre metriche statistiche

Criteri di accettazione:

- Aggiunta/rimozione dinamica di filtri statistici
- Validazione input numerici ($\text{min} < \text{max}$)
- Query ottimizzate tramite JPA Specifications
- Performance: query complesse <2 secondi

Pattern utilizzati: Specification Pattern, Strategy Pattern, Factory Method

Componenti: PlayerSpecificationFactory, StatFilterCriteria, PlayerFilterRequest

3.5 RF-05: Sistema di Colonne Dinamiche

Descrizione: Il sistema deve permettere la selezione dinamica delle colonne visualizzate, organizzate per categorie tematiche.

Categorie disponibili:

- Basic Info (Nome, Età, Posizione, Squadra)
- Playing Time (Matches, Minutes, 90s)
- Standard Attacking (Goals, Assists, G+A)
- Expected Metrics (xG, npG, xAG)
- Shooting (Shots, SoT, SoT%)
- Passing (Key Passes, Passes Completed)
- Defensive Actions (Tackles, Interceptions, Blocks)
- Per 90 Minutes statistics

Criteri di accettazione:

- Selezione multipla di categorie
- Persistenza preferenze (opzionale)
- Ordinamento colonne drag-and-drop
- Larghezza colonne adattiva

3.6 RF-06: Apertura Automatica Browser

Descrizione: Il sistema deve tentare l'apertura automatica del browser all'avvio dell'applicazione per migliorare l'esperienza utente.

Strategie implementate:

- Java Desktop API (standard)
- Comandi OS nativi (Windows, Mac, Linux fallback)
- Gestione elegante dei fallback

Criteri di accettazione:

- Tentativo multiplo con fallback graduale
- Log dettagliato del processo
- UI messaggio chiaro in caso di fallimento totale

Componenti: BrowserLauncher (@Component, @EventListener)

4 Requisiti Non Funzionali

4.1 RNF-01: Usabilità

- **Target:** Scout e analisti sportivi (non necessariamente tecnici)
- **Learnability:** Interfaccia apprendibile in <15 minuti
- **Efficiency:** Operazioni comuni completabili in <3 click
- **Error Prevention:** Validazione in-line e messaggi chiari
- **Soddisfazione:** Feedback positivo da utenti test

4.2 RNF-02: Prestazioni

- **Tempo di avvio:** <10 secondi (con caricamento dati)
- **Risposta UI:** <200ms per operazioni semplici
- **Query complesse:** <2 secondi per filtri multipli
- **Concorrenza:** Supporto per 10+ utenti simultanei
- **Memoria:** Utilizzo <512MB heap per dataset tipico

4.3 RNF-03: Affidabilità

- **Disponibilità:** 99% uptime durante sessioni lavoro
- **Fault Tolerance:** Gestione graceful di errori CSV
- **Data Integrity:** Validazione completa dati input
- **Recovery:** Ripristino automatico da errori non critici
- **Logging:** Tracciamento completo operazioni sistema

4.4 RNF-04: Manutenibilità

- **Modularità:** Architettura a 3 layer ben separati
- **Testabilità:** Code coverage >80% sui servizi core
- **Documentazione:** Javadoc completo + diagrammi UML
- **Estensibilità:** Facile aggiunta nuove metriche/filtri
- **Refactoring:** Debt tecnico <5% (SonarLint metrics)

4.5 RNF-05: Portabilità

- **Platform:** Java 17+ su qualsiasi OS (Windows, Mac, Linux)
- **Deployment:** JAR eseguibile standalone
- **Browser:** Compatibilità Chrome 90+, Firefox 88+, Safari 14+
- **Database:** H2 (dev), PostgreSQL/MySQL ready (prod)
- **Dependencies:** Gestione centralizzata via Maven

4.6 RNF-06: Sicurezza (Future Enhancement)

- **Input Validation:** Sanitizzazione dati CSV
- **SQL Injection:** Prevention via JPA/Hibernate
- **Logging:** Nessun dato sensibile nei log
- **AuthN/AuthZ:** Ready per implementazione futura

5 Requisiti di Dati

5.1 RD-01: Struttura Dataset

- **Formato:** CSV con header, encoding UTF-8
- **Separatore:** Virgola (,)
- **Valori mancanti:** "NaN", stringa vuota, "null"
- **Colonne:** 100+ attributi statistici organizzati
- **Dimensione:** Tipicamente 1-10MB per stagione

5.2 RD-02: Modello Entity

- **Player Entity:** 100+ campi con annotazioni JPA
- **Mapping:** @Column per nomi DB diversi da Java
- **Tipi:** Integer, Double, String principalmente
- **Relazioni:** Attualmente nessuna (entity standalone)
- **Indici:** @Index su campi ricercati frequentemente

5.3 RD-03: Persistenza

- **Database:** H2 in-memory per sviluppo/test
- **ORM:** Spring Data JPA con Hibernate
- **Transazioni:** @Transactional su servizi
- **Migration:** Schema generato automaticamente
- **Backup:** Export/import CSV integrato

6 Vincoli di Sistema

6.1 VS-01: Tecnologici

- **Linguaggio:** Java 17 o superiore
- **Framework:** Spring Boot 3.x, Vaadin 24
- **Build Tool:** Apache Maven
- **Database:** Compatibile JPA (H2 minimo)
- **IDE:** Eclipse con plugin Papyrus per UML

6.2 VS-02: Sviluppo

- **Version Control:** GitHub con GitFlow semplificato
- **Code Style:** Google Java Style Guide adattato
- **Testing:** JUnit 5, Mockito, SpringBootTest
- **CI/CD:** GitHub Actions (future enhancement)
- **Documentation:** Maven Site, Javadoc, UML

6.3 VS-03: Architetture

- **Pattern:** MVC, Repository, Dependency Injection
- **Layers:** Presentation, Business, Persistence
- **Modularità:** Package per responsabilità
- **Scalability:** Stateless design per orizzontale scaling

7 Software Quality

7.1 Obiettivi di Qualità

Qualità	Obiettivo	Metrica
Functional Suitability	Copertura requisiti funzionali	100% RF implementati
Performance Efficiency	Tempo risposta query	<2s per query complesse
Usability	Facilità apprendimento	<15min per operazioni base
Reliability	Uptime durante sessione	99% fault tolerance
Maintainability	Modularità codice	Cyclomatic complexity <10
Portability	Multi-piattaforma	Java 17+ su 3 major OS
Security	Input validation	0 vulnerabilità SonarLint

Tabella 3: Obiettivi di qualità software

7.2 Relazione tra Qualità e Progettazione

- **Modularità → Manutenibilità:** Package separati per layer
- **Pattern → Estensibilità:** Factory e Strategy per nuovi filtri
- **Caching → Performance:** Query ottimizzate via Specifications
- **Validation → Affidabilità:** Validazione a multi-livello
- **Documentation → Manutenibilità:** Javadoc + UML completo
- **Testing → Qualità:** Coverage >80% su servizi core

8 Modellazione dei Requisiti

8.1 Diagrammi UML Prodotti

- **Use Case Diagram:** Attori (Scout, Admin) e casi d'uso principali
- **Class Diagram:** 10+ classi organizzate in package
- **Package Diagram:** Organizzazione moduli com.scouting.*
- **Component Diagram:** Architettura a componenti Spring
- **Activity Diagram:** Flusso ricerca e filtraggio

- **Sequence Diagram:** Interazioni startup e filtri
- **Communication Diagram:** Collaborazione oggetti runtime
- **State Machine Diagram:** Stati sistema e transizioni

8.2 Tracciamento Requisiti

Requisito	Diagramma UML	Classe Implementativa
RF-01	Activity, Sequence	DataProcessingService
RF-02	Class, Component	MainView, Grid configuration
RF-03	Sequence, Communication	ScoutingService, PlayerRepository
RF-04	Class, Sequence	PlayerSpecificationFactory
RF-05	Component	MainView column system
RF-06	Sequence	BrowserLauncher
RNF-01	Use Case, Activity	UI/UX design complessivo
RNF-04	Class, Package	Package structure, SOLID principles

Tabella 4: Tracciamento requisiti → design → implementazione

9 Conclusioni

Questo documento fornisce una specifica completa e verificabile dei requisiti di **Scouting Suite**. La documentazione:

- Definisce chiaramente **cosa** il sistema deve fare (requisiti funzionali)
- Specifica **come** deve farlo (requisiti non funzionali)
- Descrive **vincoli e qualità** attese
- Fornisce base per **testing e validazione**
- Allinea aspettative tra **team di sviluppo e stakeholder**

Il documento costituirà il riferimento principale durante lo sviluppo, il testing e la valutazione finale del progetto secondo i criteri del corso di Ingegneria del Software.